

EduCore

Proyecto 2

Curso: Programación III — ISI-BSI-09

Integrante: Daniel Cubero

Grupo: Trabajo individual

Instrumentos: NetBeans y Git Bash

Fecha: 9 de agosto de 2026

Tag de entrega: **v2.0-p2**

1. Distribución de tareas

El proyecto fue realizado de forma individual. Daniel Cubero estuvo a cargo de la totalidad de las tareas de implementación, integración, corrección, pruebas y entrega.

Área	Responsable
Persistencia SQL de Empleados	Daniel Cubero
Persistencia SQL de Edificios y Aulas	Daniel Cubero
Persistencia SQL de Secciones	Daniel Cubero
Matrícula por lote	Daniel Cubero
Reportes	Daniel Cubero
Validaciones y control de cupo	Daniel Cubero
Integración Docker / MariaDB	Daniel Cubero
Pruebas manuales y entrega Git	Daniel Cubero

2. Evidencia de verificación manual (RNF-02)

Se realizaron pruebas mediante Git Bash utilizando curl contra la API del sistema y jq para inspeccionar las respuestas JSON. También se verificó el comportamiento de Docker y la persistencia después de reiniciar los contenedores. **2.1**

Flujos verificados

- Estudiantes: consulta de los estudiantes persistidos, incluyendo estudiante regular y becado.
- Empleados: consulta de empleados persistidos.
- Edificios y aulas: consulta del edificio y sus aulas persistidas.
- Secciones: consulta de secciones con docente, aula y estudiantes matriculados.
- Actualización y eliminación de sección: se verificó la actualización de una sección y posteriormente su eliminación.
- Matrícula individual: se agregó un estudiante a una sección mediante el endpoint correspondiente.
- Desmatrícula: se eliminó un estudiante de una sección y se verificó que desapareciera de la respuesta.
- Matrícula por lote: se procesó un CSV/contenido de matrícula y se verificó una respuesta exitosa.
- Control de duplicados: un segundo intento de matricular al mismo estudiante produjo un error de matrícula duplicada.
- Control de cupo: una sección con aula de capacidad 1 rechazó el segundo estudiante con el mensaje de que no había cupo disponible.
- Persistencia: después de docker compose down y docker compose up -d, estudiantes, empleados, edificios, aulas, secciones y matrículas conservaron sus datos.
- Reportes: POST /api/reporte devolvió HTTP 200 y generó reporte.txt con los conteos del sistema.

2.2 Cuatro casos de error de matrícula

Caso	Resultado observado
Carnet/estudiante inexistente	Error: "No existe estudiante con ID 999." / "No existe estudiante con carnet 999999999999."
Sección inexistente	Error: "No existe la sección." / "No existe sección con código SEC-INEXISTENTE."
Cupo lleno	Error: "La sección SEC-TEST no tiene cupo disponible."
Matrícula duplicada	Error: estudiante ya matriculado / duplicate entry al intentar insertar nuevamente.

También se verificaron errores de datos inexistentes para docente y aula, así como validaciones de campos obligatorios y capacidad inválida.

3. Instrucciones de uso

3.1 Levantar el sistema

Abrir Git Bash en la carpeta raíz del proyecto y ejecutar **docker compose up -d**. La aplicación queda disponible en el puerto 8080 y MariaDB se ejecuta dentro de Docker. Para comprobar los servicios se puede ejecutar **docker compose ps**; la base de datos debe aparecer como healthy y los servicios app, matricula y reportes como activos.

3.2 Operación de la SPA

La SPA ya forma parte del proyecto y es servida por el API en **http://localhost:8080/**. Desde la interfaz se pueden utilizar los módulos existentes de administración. El flujo esperado es trabajar con los formularios y listados de cada módulo y comprobar que los registros creados o actualizados aparecen nuevamente en los listados.

Flujo	Uso esperado
Estudiantes	Utilizar el módulo de estudiantes para registrar, consultar, actualizar o eliminar estudiantes y verificar sus datos y matrícula calculada.
Empleados	Utilizar el módulo de empleados para gestionar los empleados/docentes y comprobar que aparecen en los listados.
Edificios y aulas	Crear o consultar edificios y, dentro de ellos, gestionar aulas indicando número, capacidad y tipo.
Secciones	Crear o consultar una sección indicando código, nombre, docente y aula. La sección muestra sus estudiantes matriculados.
Matrícula	Desde el flujo de matrícula, cargar el CSV solicitado por el sistema. El backend procesa los registros y devuelve el resultado; si existe un error, la operación rechaza el registro correspondiente.
Reportes	Utilizar la opción de reporte para generar el archivo de texto. El resultado contiene los conteos de estudiantes, empleados, secciones, aulas y matrículas.

Nota: la documentación describe los flujos de la SPA de acuerdo con el alcance del enunciado. Las pruebas de verificación realizadas durante el desarrollo se ejecutaron además directamente contra los endpoints HTTP para validar el backend.

4. Persistencia, Docker y entrega

Se verificó que los datos persistidos permanecen disponibles después de reiniciar el entorno con **docker compose down** y posteriormente **docker compose up -d**. Se comprobó la recuperación de estudiantes, empleados, edificios, aulas, secciones y matrículas.

El reporte probado devolvió: Estudiantes: 3, Empleados: 1, Secciones: 3, Aulas: 3 y Matrículas: 5.

La entrega Git utiliza el tag anotado **v2.0-p2**. El repositorio quedó sincronizado con GitHub.

5. Uso de IA

Durante el desarrollo se utilizó ChatGPT como herramienta de asistencia técnica. Se empleó para analizar errores de compilación y ejecución, proponer correcciones de validación, revisar flujos de API, sugerir comandos de Git/Docker y ayudar a organizar las pruebas de verificación.

Decisiones del desarrollador: Daniel Cubero revisó y decidió qué cambios aplicar, ejecutó las pruebas, verificó las respuestas del sistema y mantuvo el control del repositorio. Las correcciones se incorporaron únicamente después de comprobar su comportamiento.

Aprendizajes: se reforzó el manejo de validaciones null, control de capacidad de aulas, persistencia SQL, interacción HTTP/JSON, uso de Docker Compose, pruebas con curl/jq y flujo de entrega con commits y tags.

Qué se haría diferente: realizar una planificación inicial más detallada de las pruebas y mantener desde el comienzo una rutina de formato y verificación antes de cada commit para reducir correcciones al final.

Código externo: no se incorporó código externo de terceros como dependencia funcional del proyecto. Las herramientas utilizadas para el desarrollo y la verificación fueron las indicadas anteriormente y las dependencias definidas por el proyecto.

6. Cierre

El proyecto fue desarrollado individualmente y quedó preparado para la entrega correspondiente al Proyecto 2, incluyendo persistencia SQL, matrícula por lote, reportes, validaciones, control de cupo, contenedores Docker y documentación.